LAPORAN PRAKTIKUM
PEMROGRAMAN 3 MINGGU KE-9
DOCKER VOLUME & NETWORK



Muhammad Faiqul Umam
NRP: 3325600070
Dosen Pengampu : Dian Septiani Santoso, M. Kom

PROGRAM STUDI D4 SAINS DATA TERAPAN
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

HASIL PRAKTIKUM

Docker Volume

Latihan

Tujuan: membuktikan satu volume bisa dipakai bersama oleh dua container, dan datanya tetap ada setelah container dihapus

1. Langkah 1: Membuat volume
   membuat tempat penyimpanan bernama shared_data yang dikelola Docker dan terpisah dari container

```
3325600070_Linux@MSI:~$ docker volume create shared_data
shared_data
```

2. Langkah 2: Memastikan volume terbuat
   menampilkan daftar semua volume

```
3325600070_Linux@MSI:~$ docker volume ls
DRIVER    VOLUME NAME
local     data_volume
local     mysql_data
local     shared_data
local     student-risk-web_student-risk-data
```

3. Langkah 3: Menjalankan container ubuntu_a
   - -itd: container berjalan di latar belakang
   - --name ubuntu_a: memberi nama container
   - -v shared_data:/app: menempelkan volume shared_data ke folder /app di dalam container
   - sleep infinity: menjaga container tetap hidup, karena Ubuntu akan langsung berhenti jika tidak ada proses yang berjalan

```
3325600070_Linux@MSI:~$ docker run -itd --name ubuntu_a -v shared_data:/app ubuntu sleep infinity
1b90484e459edcf80f33d180f8942b7bc9ea0f7b7016568b82e5eb424cdf25e7
```

4. Langkah 4: Menjalankan container ubuntu_b
   sama seperti langkah 3, tetapi container kedua dengan volume yang sama. Folder /app di kedua container berisi data yang sama

```
3325600070_Linux@MSI:~$ docker run -itd --name ubuntu_b -v shared_data:/app ubuntu sleep infinity
d87a4cf1f45bf374b619e6b36e85ce93d0eabc7544cc2c334efe2c1d7520ab9d
```

5. Langkah 5: Membuat file dari ubuntu_a
   docker exec menjalankan perintah di dalam container yang sedang berjalan. Perintah ini membuat file hello.txt di /app, sehingga file tersimpan di volume

```
3325600070_Linux@MSI:~$ docker exec ubuntu_a bash -c 'echo "Halo dari ubuntu_a" > /app/hello.txt'
```

6. Langkah 6: Menampilkan isi file dari ubuntu_b
   membaca file yang dibuat oleh ubuntu_a dari container lain

```
3325600070_Linux@MSI:~$ docker exec ubuntu_b cat /app/hello.txt
Halo dari ubuntu_a
```

7. Langkah 7: Menghapus kedua container
   menghapus paksa kedua container. Volume tidak ikut terhapus

```
3325600070_Linux@MSI:~$ docker rm -f ubuntu_a ubuntu_b
ubuntu_a
ubuntu_b
```

8. Langkah 8: Membuat container baru dengan volume yang sama

```
3325600070_Linux@MSI:~$ docker run --rm -v shared_data:/app ubuntu cat /app/hello.txt
Halo dari ubuntu_a
```

Tugas

Tujuan: membuat aplikasi Flask yang menyimpan data ke PostgreSQL, lalu membuktikan data tetap ada setelah docker compose down dan up

1. Langkah 1: Membuat folder kerja

   membuat dan masuk ke folder khusus agar file rapi dan docker-compose.yml ditemukan oleh Compose

```
3325600070_Linux@MSI:~$ mkdir praktikum-volume
3325600070_Linux@MSI:~$ cd praktikum-volume
3325600070_Linux@MSI:~/praktikum-volume$
```

2. Langkah 2: Membuat requirements.txt

   daftar library Python. flask untuk web server, psycopg2-binary agar Python bisa terhubung ke PostgreSQL

```
3325600070_Linux@MSI:~/praktikum-volume$ nano requirements.txt
  GNU nano 7.2
flask
psycopg2-binary
```

3. Langkah 3: Membuat app.py
   - get_conn(): membuka koneksi ke database. host="db" memakai nama service db dari Compose sebagai alamat (DNS internal Docker)
   - init_db(): membuat tabel users saat aplikasi mulai, dengan percobaan ulang hingga 30 kali karena Postgres butuh beberapa detik untuk siap
   - /add/<name>: memasukkan nama ke tabel
   - /users: menampilkan seluruh isi tabel
   - /: halaman info

```
3325600070_Linux@MSI:~/praktikum-volume$ nano app.py
```

```
import os
import time
import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)

def get_conn():
    return psycopg2.connect(
        host="db",
        dbname=os.environ["POSTGRES_DB"],
        user=os.environ["POSTGRES_USER"],
        password=os.environ["POSTGRES_PASSWORD"],
    )

def init_db():
    for _ in range(30):
        try:
            conn = get_conn()
            cur = conn.cursor()
            cur.execute("CREATE TABLE IF NOT EXISTS users (id SERIAL PRIMARY KEY, name TEXT)")
            conn.commit()
            conn.close()
            return
        except Exception:
            time.sleep(2)

@app.route("/")
def index():
    return "Flask + PostgreSQL jalan. Pakai /add/<nama> dan /users"

@app.route("/add/<name>")
def add_user(name):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("INSERT INTO users (name) VALUES (%s)", (name,))
    conn.commit()
    conn.close()
    return jsonify({"status": "ok", "added": name})
```

```
@app.route("/users")
def list_users():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("SELECT id, name FROM users ORDER BY id")
    rows = cur.fetchall()
    conn.close()
    return jsonify([{"id": r[0], "name": r[1]} for r in rows])

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000)
```

4. Langkah 4: Membuat Dockerfile
   - FROM: memakai image dasar Python 3.12
   - WORKDIR: menentukan folder kerja /app di dalam container
   - COPY requirements.txt . dan RUN pip install: menyalin dan memasang dependensi
   - COPY app.py .: menyalin kode aplikasi
   - CMD: perintah yang dijalankan saat container start

```
3325600070_Linux@MSI:~/praktikum-volume$ nano Dockerfile
  GNU nano 7.2
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

5. Langkah 5: Membuat docker-compose.yml
   - Service db: memakai image postgres:16. environment mengatur user, password, dan nama database. volumes: pg_data:/var/lib/postgresql/data menyimpan data database ke volume pg_data.
   - Service web: dibangun dari Dockerfile (build: .), port 5000 dibuka ke komputer, dan depends_on membuat db dijalankan lebih dulu.
   - volumes: pg_data: di bawah: mendeklarasikan volume

```
3325600070_Linux@MSI:~/praktikum-volume$ nano docker-compose.yml
  GNU nano 7.2
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: 12345
      POSTGRES_DB: appdb
    volumes:
      - pg_data:/var/lib/postgresql/data

  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: 12345
      POSTGRES_DB: appdb
    depends_on:
      - db

volumes:
  pg_data:
```

6. Langkah 6: Memeriksa file

```
3325600070_Linux@MSI:~/praktikum-volume$ ls
Dockerfile  app.py  docker-compose.yml  requirements.txt
```

7. Langkah 7: Menjalankan semua service
   membangun image Flask lalu menjalankan service db dan web di latar belakang (-d)

```
3325600070_Linux@MSI:~/praktikum-volume$ docker compose up -d --build
[+] up 21/21
 ✔Image postgres:16                  Pulled
 ✔Image praktikum-volume-web         Built
 ✔Network praktikum-volume_default   Created
 ✔Volume praktikum-volume_pg_data    Created
 ✔Container praktikum-volume-db-1    Started
 ✔Container praktikum-volume-web-1   Started
```

8. Langkah 8: Memeriksa status container
   menampilkan status service

```
3325600070_Linux@MSI:~/praktikum-volume$ docker compose ps
NAME                    IMAGE                   COMMAND                  SERVICE   CREATED              STATUS
    PORTS
praktikum-volume-db-1    postgres:16             "docker-entrypoint.s…"   db        About a minute ago   Up About a minut
e   5432/tcp
praktikum-volume-web-1   praktikum-volume-web    "python app.py"          web       About a minute ago   Up About a minut
e   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

9. Langkah 9: Menambah data dari aplikasi
   mengakses aplikasi untuk memasukkan dua data user ke database

```
3325600070_Linux@MSI:~/praktikum-volume$ curl localhost:5000/add/budi
{"added":"budi","status":"ok"}
3325600070_Linux@MSI:~/praktikum-volume$ curl localhost:5000/add/siti
{"added":"siti","status":"ok"}
```

10. Langkah 10: Menampilkan data sebelum container dihentikan

```
3325600070_Linux@MSI:~/praktikum-volume$ curl localhost:5000/users
[{"id":1,"name":"budi"},{"id":2,"name":"siti"}]
```

11. Langkah 11: Menghentikan container
    menghentikan dan menghapus container serta network. Volume tidak dihapus karena tidak memakai -v

```
3325600070_Linux@MSI:~/praktikum-volume$ docker compose down
[+] down 3/3
 ✔Container praktikum-volume-web-1 Removed
 ✔Container praktikum-volume-db-1  Removed
 ✔Network praktikum-volume_default Removed
```

12. Langkah 12: Memastikan volume masih ada

```
3325600070_Linux@MSI:~/praktikum-volume$ docker volume ls
DRIVER    VOLUME NAME
local     data_volume
local     mysql_data
local     praktikum-volume_pg_data
```

13. Langkah 13: Menjalankan ulang
    membuat container baru, tetapi menempelkan volume lama

```
3325600070_Linux@MSI:~/praktikum-volume$ docker compose up -d
[+] up 3/3
 ✔Network praktikum-volume_default Created
 ✔Container praktikum-volume-db-1  Started
 ✔Container praktikum-volume-web-1 Started
```

14. Langkah 14: Membuktikan data masih ada

```
3325600070_Linux@MSI:~/praktikum-volume$ curl localhost:5000/users
[{"id":1,"name":"budi"},{"id":2,"name":"siti"}]
```

Docker Network

Latihan

Tujuan: membuktikan container yang berada di network berbeda tidak bisa saling berkomunikasi, dan hanya container yang tersambung ke kedua network (gateway) yang bisa menjangkau keduanya

1. Langkah 1: Membuat dua network

   membuat dua jaringan bridge terpisah. Container di jaringan berbeda terisolasi satu sama lain

```
3325600070_Linux@MSI:~$ docker network create frontend_net
9a30fd7769bf1bdecd7429cd05bbcb86bd4b7c12e8a0dde00fbf5c5361d332d2
3325600070_Linux@MSI:~$ docker network create backend_net
a07f782bc3da2414faec156498a6e86d90fc81ebcf383e6c99a322df5ca9e3f3
```

2. Langkah 2: Memastikan network terbuat

   menampilkan semua network

```
3325600070_Linux@MSI:~$ docker network ls
NETWORK ID     NAME                                DRIVER    SCOPE
a07f782bc3da   backend_net                         bridge    local
4696367a7ef9   bridge                              bridge    local
9a30fd7769bf   frontend_net                        bridge    local
```

3. Langkah 3: Menjalankan app_frontend
   - --network frontend_net: container langsung dihubungkan ke frontend_net.
   - sleep infinity: menjaga container tetap hidup

```
3325600070_Linux@MSI:~$ docker run -itd --name app_frontend --network frontend_net ubuntu sleep infinity
4a82cb76726fcce740a386798608c8795af5a29a3977f9855a21f81469b5050b
```

4. Langkah 4: Menjalankan app_backend

   sama seperti langkah 3, tetapi terhubung ke backend_net

```
3325600070_Linux@MSI:~$ docker run -itd --name app_backend --network backend_net ubuntu sleep infinity
8c4cf94b4eb46a4ba91eb6299eacfbc1f58bc5621cccc8c3caa22b4b4e2b7bf0
```

5. Langkah 5: Menjalankan gateway

   gateway awalnya hanya terhubung ke frontend_net

```
3325600070_Linux@MSI:~$ docker run -itd --name gateway --network frontend_net ubuntu sleep infinity
9fcc1823320df7a7ca7268d3a56fe7df737abdb68f43e7a209e21a0332669b50
```

6. Langkah 6: Menghubungkan gateway ke backend_net

   menambahkan gateway ke network kedua. Sekarang gateway ada di dua network sekaligus

```
3325600070_Linux@MSI:~$ docker network connect backend_net gateway
```

7. Langkah 7: Memasang ping di ketiga container

   image Ubuntu minimal tidak menyertakan ping, jadi dipasang dulu di setiap container

```
3325600070_Linux@MSI:~$ docker exec app_frontend bash -c "apt update && apt install -y iputils-ping"
3325600070_Linux@MSI:~$ docker exec app_backend bash -c "apt update && apt install -y iputils-ping"
3325600070_Linux@MSI:~$ docker exec gateway bash -c "apt update && apt install -y iputils-ping"
```

8. Langkah 8: Tes ping dari gateway ke app_frontend

   -c 3 membatasi ping 3 kali. Container dipanggil dengan nama, bukan IP, berkat DNS internal Docker

```
3325600070_Linux@MSI:~$ docker exec gateway ping -c 3 app_frontend
PING app_frontend (172.20.0.2) 56(84) bytes of data.
64 bytes from app_frontend.frontend_net (172.20.0.2): icmp_seq=1 ttl=64 time=0.137 ms
64 bytes from app_frontend.frontend_net (172.20.0.2): icmp_seq=2 ttl=64 time=0.119 ms
64 bytes from app_frontend.frontend_net (172.20.0.2): icmp_seq=3 ttl=64 time=0.122 ms

--- app_frontend ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2120ms
rtt min/avg/max/mdev = 0.119/0.126/0.137/0.007 ms
```

9. Langkah 9: Tes ping dari gateway ke app_backend
   karena gateway ada di backend_net, ping ke app_backend juga berhasil

```
3325600070_Linux@MSI:~$ docker exec gateway ping -c 3 app_backend
PING app_backend (172.21.0.2) 56(84) bytes of data.
64 bytes from app_backend.backend_net (172.21.0.2): icmp_seq=1 ttl=64 time=0.144 ms
64 bytes from app_backend.backend_net (172.21.0.2): icmp_seq=2 ttl=64 time=0.074 ms
64 bytes from app_backend.backend_net (172.21.0.2): icmp_seq=3 ttl=64 time=0.096 ms

--- app_backend ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2079ms
rtt min/avg/max/mdev = 0.074/0.104/0.144/0.029 ms
```

10. Langkah 10: Tes ping dari app_frontend ke app_backend
    kedua container berada di network berbeda

```
3325600070_Linux@MSI:~$ docker exec app_frontend ping -c 3 app_backend
ping: app_backend: Name or service not known
```

11. Langkah 11: Melihat hostname dan IP semua container
    menampilkan nama container beserta IP di setiap network-nya. Untuk hostname:

```
3325600070_Linux@MSI:~$ docker inspect -f '{{.Name}}{{range $k,$v := .NetworkSettings.Networks}} {{$k}}={{$v.IPAddress}}
{{end}}' app_frontend app_backend gateway
/app_frontend frontend_net=172.20.0.2
/app_backend backend_net=172.21.0.2
/gateway backend_net=172.21.0.3 frontend_net=172.20.0.3
```

12. Langkah 12: Membersihkan
    menghapus container dulu, baru network (network yang masih dipakai container tidak bisa dihapus

```
3325600070_Linux@MSI:~$ docker rm -f app_frontend app_backend gateway
app_frontend
app_backend
gateway
3325600070_Linux@MSI:~$ docker network rm frontend_net backend_net
frontend_net
backend_net
```

Tugas

Tujuan: menjalankan dua service (web dan db) dalam satu network khusus app_network dengan Docker Compose, lalu membuktikan keduanya dapat berkomunikasi memakai nama service

1. Langkah 1: Membuat folder kerja
   folder khusus agar docker-compose.yml mudah ditemukan Compose

```
3325600070_Linux@MSI:~$ mkdir praktikum-network
3325600070_Linux@MSI:~$ cd praktikum-network
3325600070_Linux@MSI:~/praktikum-network$
```

2. Langkah 2: Membuat docker-compose.yml
   - Service web: memakai image Nginx, port 80 container dibuka ke port 8080 komputer, dan dihubungkan ke app_network.
   - Service db: memakai image MySQL 8, password root diatur lewat environment, dan dihubungkan ke app_network.
   - networks: app_network di bawah: mendeklarasikan network bridge khusus

```
3325600070_Linux@MSI:~/praktikum-network$ nano docker-compose.yml
```

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    networks:
      - app_network

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: 12345
    networks:
```

```
    - app_network

networks:
  app_network:
    driver: bridge
```

3. Langkah 3: Memeriksa file

```
3325600070_Linux@MSI:~/praktikum-network$ ls
docker-compose.yml
```

4. Langkah 4: Menjalankan service

mengunduh image, membuat network app_network, lalu menjalankan web dan db di latar belakang

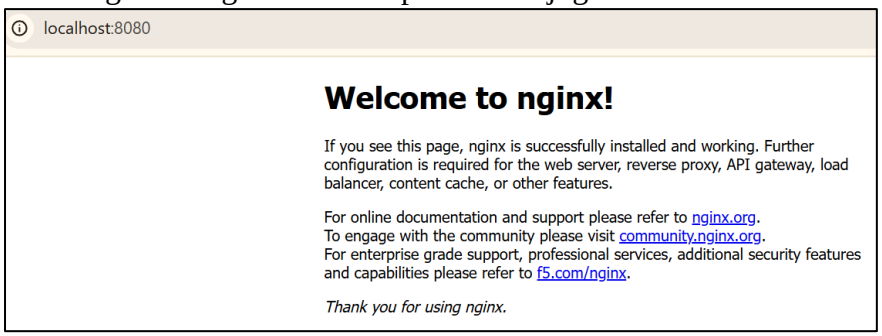
```
3325600070_Linux@MSI:~/praktikum-network$ docker compose up -d
[+] up 12/12
 ✔Image nginx                              Pulled
 ✔Network praktikum-network_app_network    Created
 ✔Container praktikum-network-web-1        Started
 ✔Container praktikum-network-db-1         Started
```

5. Langkah 5: Memeriksa status container

```
3325600070_Linux@MSI:~/praktikum-network$ docker compose ps
NAME                      IMAGE     COMMAND                  SERVICE   CREATED          STATUS          PORTS
praktikum-network-db-1    mysql:8   "docker-entrypoint.s…"   db        27 seconds ago   Up 26 seconds   3306/tcp, 33060/
tcp
praktikum-network-web-1   nginx     "/docker-entrypoint.…"   web       27 seconds ago   Up 26 seconds   0.0.0.0:8080->80
/tcp, [::]:8080->80/tcp
```

6. Langkah 6: Mengecek web berjalan

mengakses Nginx dari komputer. Bisa juga dibuka di browser http://localhost:8080



7. Langkah 7: Memeriksa network

Compose menambahkan nama folder sebagai prefix, jadi namanya praktikum-network_app_network

```
3325600070_Linux@MSI:~/praktikum-network$ docker network ls
NETWORK ID     NAME                                DRIVER    SCOPE
4696367a7ef9   bridge                              bridge    local
6bb6bfb6b8d9   host                                host      local
2b062ec67664   my_network                          bridge    local
14ae510ff821   none                                null      local
cb0953b43401   praktikum-network_app_network       bridge    local
```

8. Langkah 8: Memastikan web dan db di network yang sama

menampilkan detail network

```
3325600070_Linux@MSI:~/praktikum-network$ docker network inspect praktikum-network_app_network
[
    {
        "Name": "praktikum-network_app_network",
        "Id": "cb0953b4340175d78179acbe747fa0c6112c6a270fc3c3421f2f0074bd69f188",
        "Created": "2026-10-21T07:57:02.193425125+07:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
```

9. Langkah 9: Memasang ping di container web

   image Nginx tidak menyertakan ping, jadi dipasang dulu. docker compose exec web menjalankan perintah di dalam service web

```
3325600070_Linux@MSI:~/praktikum-network$ docker compose exec web bash -c "apt update && apt install -y iputils-ping"
```

10. Langkah 10: Ping db dari web

    web memanggil db memakai nama service, bukan IP

```
3325600070_Linux@MSI:~/praktikum-network$ docker compose exec web ping -c 3 db
PING db (172.20.0.3) 56(84) bytes of data.
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.20.0.3): icmp_seq=1 ttl=64 time=0.158 ms
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.20.0.3): icmp_seq=2 ttl=64 time=0.095 ms
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.20.0.3): icmp_seq=3 ttl=64 time=0.087 ms

--- db ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2158ms
rtt min/avg/max/mdev = 0.087/0.113/0.158/0.031 ms
```

11. Langkah 11: Membersihkan

    menghentikan dan menghapus container serta network app_network

```
3325600070_Linux@MSI:~/praktikum-network$ docker compose down
[+] down 3/3
 ✔Container praktikum-network-web-1      Removed
 ✔Container praktikum-network-db-1       Removed
 ✔Network praktikum-network_app_network Removed
```